CNC Part Analyzer – Technical Document

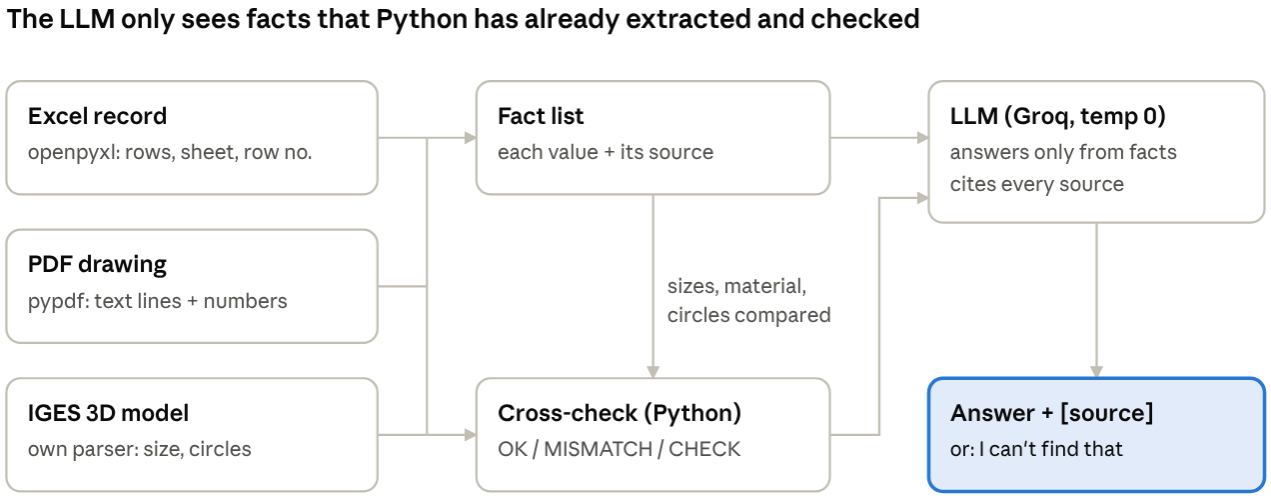
Oct 27, 2026 · @Anees Ansari

Overview

Our app takes a part number and a question, finds that part's Excel manufacturing record, PDF engineering drawing and IGES 3D model, cross-checks them, and answers in plain English with the source file of every fact. **Level reached: 3 (Excel + PDF drawing + IGES model, cross-checked), plus the stretch goal: it works for any part number in the dataset, including files it has never seen.**

- **Language:** Python 3, one file (`cnc_app.py`), runs in a terminal.
- **Libraries:** `openpyxl` (Excel), `pypdf` (PDF). The IGES reader is our own code, so no CAD software is needed.
- **LLM:** Groq API (hosted, OpenAI-compatible). The app picks OpenAI gpt-oss-120b (`openai/gpt-oss-120b`) (the model it chose on our test run) and prints the model name at start.
- **Rule we built around:** the LLM never reads the files. Python extracts the facts; the LLM only explains them. If a fact is missing, the app says *"I can't find that in the files."*

Architecture



data flow · 3 readers, 1 cross-check, 1 LLM

The user types a part number and a question. Three readers turn the files into one fact list, Python compares the files, and only then does the LLM write the answer, citing the source of each fact.

How each file type is read

Each reader turns its file into a list of facts. Every fact carries its exact source (file + sheet/row, page, or model), which is what lets every answer cite its evidence.

File	Library	What we extract	Source label kept
Excel (.xlsx / .csv)	openpyxl, csv	Every row on every sheet that contains the part number, with its column titles	File, sheet, row number
PDF drawing (.pdf)	pypdf	Every text line and every number; title-block lines (material, finish, tolerance, units) are marked	File, page
IGES model (.igs / .iges)	Our own parser	Units, overall size (X × Y × Z), diameters of full circles, list of shape types	File

Excel. The app finds the column-title row by itself (many records have a title above the table), then matches the part number exactly; if that fails it accepts forms like *P-2280* or *2280-A*. All sheets are searched, so a second sheet such as a routing plan is included.

PDF drawing. Text is read page by page. A label and its value printed separately ("MATERIAL:" then "ALUMINIUM 6061-T6") are joined into one fact. A page with no text is reported as a probable scanned image instead of being guessed.

IGES model. IGES is a plain-text CAD format: the Global section gives the units, the Directory section lists every entity and the Parameter section holds its numbers. Our parser:

1. Reads lines (110), circular arcs (100) and B-spline curves (126, evaluated with the de Boor algorithm).
2. Applies each entity's transformation matrix (124), because real CAD exporters place arcs and curves with one.
3. Keeps only real edges: those listed by edge lists (504) in solid files or by curve-on-surface entities (142) in surface files. Helper curves such as the axis of a round face are skipped.
4. Reports the bounding box of those edges as the overall size, and full circles as hole or round-face diameters.

Tested on IGES files written by Open CASCADE in both surface mode and solid (B-rep) mode: the sizes matched the true CAD sizes exactly for all five sample parts.

Cross-checking

The three sources are compared in plain Python, not by the LLM, so the checks are repeatable. Each result is **OK**, **MISMATCH** (the files disagree) or **CHECK** (could not be confirmed); all results are shown on screen and passed to the LLM.

Check	Compares	Tolerance
Part number on drawing	Part number vs drawing text	Exact
Material	Excel material vs drawing title block; the drawing's value is quoted on a mismatch	First two words, e.g. *aluminium 6061*
Excel sizes vs drawing	Each Length / Width / Height / Diameter column vs drawing numbers	±0.5 mm
3D size vs drawing	Model X, Y, Z size (converted to mm) vs drawing numbers	±0.5 mm
Excel sizes vs 3D model	Excel sizes vs model size, both sorted largest first	±0.5 mm
Circles vs drawing	Each full-circle diameter in the model vs drawing numbers	±0.05 mm

Finding the right files for a part is also checked. The app uses, in order: a file name containing the part number, a file named in the part's Excel row, the part number found inside the file, or the only file of that type in the folder. The last case is shown as a warning and passed to the LLM, so it is never presented as confirmed.

How the LLM is used

The LLM only turns checked facts into a short, plain-English answer. It never sees the raw files, so it cannot invent values the files do not contain.

- **Provider and model:** Groq API (hosted). The app lists the models on the account and picks OpenAI gpt-oss-120b (`openai/gpt-oss-120b`) (the model it chose on our test run); the model name is printed when the app starts.
- **Settings:** temperature 0, so the same question gives the same answer.
- **What it receives:** the part number, every fact as `[source] field: value`, the cross-check results, any missing-file notes, and the question.

- **System rules:** (1) answer only from the facts, no outside knowledge, no guessed numbers; (2) put the source in brackets after every claim; (3) if the facts do not contain the answer, say *"I can't find that in the files"* and say what is available; (4) mention any MISMATCH or CHECK related to the question; (5) keep it short and give units.
- **Odd questions:** a question unrelated to the part, or about data that is not in the files (for example, price), is answered with *"I can't find that in the files."*
- **Offline fallback:** if the API cannot be reached, the app still answers by showing the facts that match the question's words, with their sources, or says it cannot find the answer.
- **Evidence on request:** typing `facts` prints every fact the LLM was given, so judges can check any answer against the files.

Sample dataset and test results

The organizers asked teams to find their own reference files, so we built a 5-part dataset: one Excel record (2 sheets: *Manufacturing Record* with a title above the table, and *Routing*), 5 PDF drawings (front, top, side and isometric views, dimensions, notes, title block) and 5 IGES models. Part 2284's 3D model is a public sample part from the open-source pythonocc-demos repository on GitHub; parts 2280–2283 were modelled in CadQuery (open-source CAD). Two faults were planted on purpose to test the app.

Part	Name	3D size found (mm)	Circle diameters found (mm)	Cross-check result
2280	Mounting Bracket	100 × 60 × 20	10	All OK
2281	Pipe Flange	80 × 80 × 15	8, 30, 80	All OK
2282	Stepped Shaft	120 × 25 × 25	18, 23, 25	OK; CHECK on Ø23 (chamfer edge, not dimensioned). Surface finish left out of Excel on purpose
2283	Spacer Block	50 × 40 × 30	12	**MISMATCH (planted):** Excel says Brass CZ121, drawing says Aluminium 7075-T6
2284	Machined Test Block	315 × 105 × 225	46.257	OK on sizes; CHECK on Ø46.257 (not on drawing)

Robustness tests run: files renamed so no name contains the part number; the Excel saved as CSV with the part written as *P-2280*; an unknown part number (9999), which gives a clean "not found" message; the AI unreachable, which switches to offline mode; and the app run with the heavy libraries (pandas, pdfplumber) blocked, as on a locked-down Windows laptop. All passed.

Limitations

- **Size is a bounding box** along the model's X, Y and Z axes. A part modelled at an angle gives a larger box than its true size.
- **Circles are not guaranteed holes.** A full circle can also be a round outer face or a chamfer edge, so the app reports "circle edges", and a through-hole usually appears twice (one at each end).
- **Older IGES curve types** (112 and 114 parametric splines) are not measured; the app says the size could not be measured instead of guessing.
- **Scanned PDF drawings** have no text to read. The app flags them; facts can be typed into a `<drawing name>_notes.txt` file, labelled in every answer as typed by the team.
- **Drawing numbers are matched as a list**, without knowing which dimension line each belongs to, so a size can match an unrelated number.
- **File formats:** .xls (old Excel) and STEP models are not read; save as .xlsx/.csv and export the model as IGES.
- **The LLM can still misread a fact.** The app always shows the cross-check and can print every fact it sent, so an answer can be checked against the files.

How to run

1. Install Python 3.10 or newer, then the two libraries: `py -m pip install openpyxl pypdf`
2. Put the Excel, PDF and IGES files in the `data` folder (the sample dataset is already there).
3. Run `py cnc_app.py`, paste a free Groq API key the first time, type a part number, then ask questions. Type `facts` to see the evidence, `new` for another part, `quit` to exit.
4. Without the LLM: `py cnc_app.py --check 2280` prints the files found, the facts and the cross-check.
5. Other files (for example from the judges): `py cnc_app.py --data E:\` (any folder).

Team

Team Peace Invaders: Ahsan (team lead), Basit and Ezan.

Sources

- pythonocc-demos: source of the part 2284 3D model (`face_recognition_sample_part`)
- CadQuery: open-source CAD used to model parts 2280–2283 and export IGES
- Groq API: hosted LLM access
- openpyxl and pypdf: Excel and PDF reading